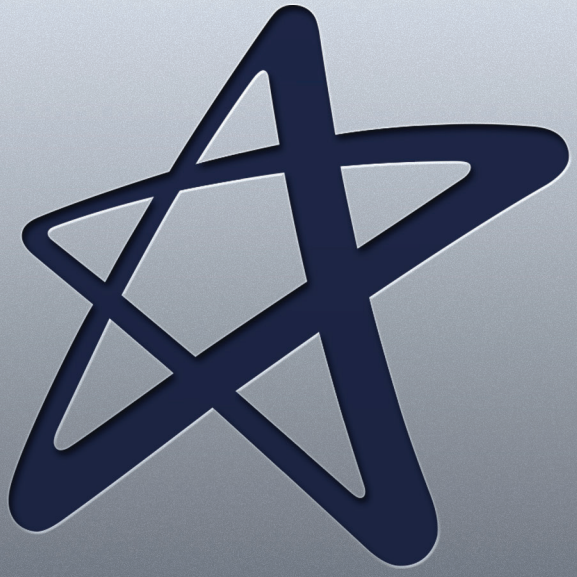
Pós-Graduação

Banco de Dados Open Source



Cruzeiro do Sul Virtual
Educação a Distância

Introdução Sobre Banco de Dados *Open Source*

Responsável pelo Conteúdo:
Prof. Me. Luiz Carlos Reis

Revisão Textual:
Prof. Me. Luciano Vieira Francisco

Unidade

Introdução Sobre Banco de Dados *Open Source*

Nesta unidade, trabalharemos os seguintes tópicos:

- Introdução;
- Bancos de Dados Relacionais;
- O que Motivou o Surgimento do NoSQL?
- O que são Bancos de Dados NoSQL?
- Qual Solução de Bancos de Dados Devemos Utilizar?
- O que Pode Motivar a Utilização de Banco de Dados Open Source?
- Bancos de Dados Relacionais Versus NoSQL: Qual Escolher?
- NoSQL é Mais Propício para Big Data;
- Conclusão.



Fonte: iStock/Getty Images

Objetivos

- Conhecer quais bancos de dados open source existem no mercado e a diferença entre banco de dados open source relacionais e não relacionais;
- Identificar as vantagens e desvantagens na utilização de banco de dados não relacionais.

Caro Aluno(a)!

Normalmente, com a correria do dia a dia, não nos organizamos e deixamos para o último momento o acesso ao estudo, o que implicará o não aprofundamento no material trabalhado ou, ainda, a perda dos prazos para o lançamento das atividades solicitadas.

Assim, organize seus estudos de maneira que entrem na sua rotina. Por exemplo, você poderá escolher um dia ao longo da semana ou um determinado horário todos ou alguns dias e determinar como o seu "momento do estudo".

No material de cada Unidade, há videoaulas e leituras indicadas, assim como sugestões de materiais complementares, elementos didáticos que ampliarão sua interpretação e auxiliarão o pleno entendimento dos temas abordados.

Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois estes ajudarão a verificar o quanto você absorveu do conteúdo, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e aprendizagem.

Bons Estudos!

Introdução

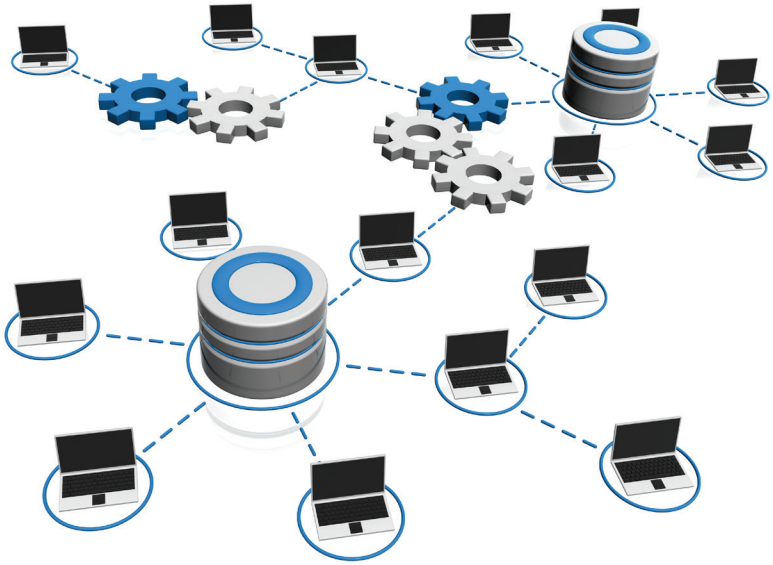


Figura 1

Cada vez mais os *softwares* livres vêm sendo utilizados nos dias atuais, e se tratando de banco de dados não poderia ser diferente. Os *softwares* pagos já estão consolidados, caso exemplar das grandes corporações, tais como IBM, Microsoft, Oracle, entre outras.

Atualmente, os bancos de dados livres também começam a se destacar da mesma forma que aconteceu com o Linux. Antes de começarmos a mencionar *open source*, vale a pena entender a diferença entre *software* gratuito e *software open source*.

Software gratuito não tem custo para quem o utiliza, mas isso não quer dizer que possa ser usado sem restrições. A grande maioria dos bancos de dados "*express*" tem restrições quanto ao número máximo de usuários, memória, processadores, ou até mesmo se o uso é pessoal ou comercial.

Já os bancos de dados *open source* podem ser usados sem restrições e o Sistema Gerenciador de Banco de Dados (SGBD) tem o código-fonte aberto, existindo a possibilidade de visualizar, alterar e recompilar o código-fonte.

Mas, qual é o mais apropriado para utilizarmos?

Isso requer, primeiramente, analisar qual será a real utilização do banco de dados. Atualmente, os bancos de dados já não são mais simples armazenadores de informações, dado que existem vários serviços incorporados a esses *softwares*. As estruturas de dados estão cada vez mais complexas e essas devem ser armazenadas pelos bancos de dados para suprir as necessidades diversas dos usuários.

Em um passado não muito distante, a única alternativa para armazenar os dados corporativos de forma estruturada era utilizar um Sistema Gerenciador de Banco de Dados Relacionais (SGBDR). Os bancos de dados proprietários eram as únicas opções no

mercado, tais como Oracle e Microsoft SQL Server, mais os bancos de dados de licenças *open source* MySQL e PostgreSQL.

Durante muitos anos, esses SGBDR atenderam ao seu propósito e ainda o fazem muito bem, mas com o surgimento dos universos de big data, data warehouse e data mining e uma grande quantidade de dados de diferentes categorias tendo que ser armazenados e gerados em distintas velocidades e volumes, novos modelos de gestão de dados começaram a surgir. Isto levou ao crescimento de soluções NoSQL, comumente referidos como *Not Only SQL*.

Os bancos de dados não relacionais, denominados NoSQL, entraram em cena e passaram a disputar mercado com os bancos de dados relacionais com o intuito de competir com a tecnologia utilizada no armazenamento de dados corporativos. Com o NoSQL, dados não estruturados podem ser armazenados em vários nós de processamento e não requerem estruturas fixas, geralmente evitando operações com relacionamento e, normalmente, funcionando bem quando utilizam escalonamento horizontal.

Bancos de Dados Relacionais

Os bancos de dados relacionais armazenam e representam os dados em tabelas, as quais organizadas por meio de linhas e colunas. São baseados na teoria dos conjuntos algébricos, então conhecida como álgebra relacional. Ademais, utilizam a linguagem SQL – *Structured Query Language* – para acesso aos dados, tornando-os uma boa escolha para aplicações que envolvem a gestão de várias operações.

A estrutura de um banco de dados relacional permite unir informações de diferentes tabelas por meio do uso de chaves estrangeiras – ou índices – que são empregadas para identificar exclusivamente qualquer peça atômica de dados dentro dessas tabelas, as quais se conectam através de chaves estrangeiras, de modo a criar uma ligação entre as suas peças de dados.

Para a manipulação das informações nos bancos de dados relacionais, a linguagem padrão é o SQL. Em 1986, surgiu o primeiro padrão, que ficou conhecido como SQL-86. Uma nova versão foi lançada em 1989, denominada SQL-89. Logo se chegou a um padrão com regras para definir os bancos de dados relacionais – a grande maioria dos bancos de dados atualmente utilizados é compatível com essa versão. A última alteração desse padrão ocorreu em 1999, conhecida como SQL-99, que trouxe muitas mudanças, algumas das quais relacionadas à definição do padrão para banco de dados de "objetos relacionais".

Você pode usar SQL para operações em banco de dados, tais como consulta de dados, atualização, ou mesmo remoção dos dados.

Um servidor RDBMS – *Relational Database Management System* – consiste em um banco de dados relacional, que dispõe de um conjunto de tabelas que contêm dados com categorias ou colunas predefinidas. Comporta dados estruturados, tais como nomes,

endereços de e-mail, dados de vendas, contas a pagar, cadastro de clientes etc. Um banco de dados relacional combina dados usando características comuns encontradas no conjunto de dados e o grupo resultante é denominado *schema*.

Esse modelo também é utilizado pelos bancos de dados relacionais *open source*, sendo a diferença voltada à forma que a informação é tratada. Cada *software* gerencia de uma forma diferente os seus dados, usando controles de transação, replicação, garantia de integridade etc. Há aqueles mais simples por se basearem em arquivos de dados. Alguns possuem uma linguagem de programação complexa, enquanto outros não.

Contudo, sabemos que os bancos de dados usam o mesmo padrão, mas nem por isso são iguais. Os fatores que nos influenciam a adquirir determinado sistema de bancos de dados são vários, entre os quais podemos destacar:

- **Recuperação** – quando temos algum tipo de problema, como recuperamos os dados? Backup on-line em vários servidores e clusters correspondem a ferramentas realmente úteis e que devemos levar em consideração;
- **Suporte a transações** – é preciso realizar o controle dessas transações, garantindo integridade das informações quando há vários usuários manipulando-as simultaneamente. Atualmente é necessário, pelo menos, o bloqueio por linha, isto é, cada operação bloqueará uma linha (registro) no banco de dados;
- **Suporte à programação** – o banco de dados deve ter uma linguagem de programação que permita realizar rotinas específicas sobre o qual;
- **Desempenho** – o banco de dados deve ser rápido, em um desempenho que possa ser melhorado por meio de técnicas de ajustes realizados no próprio banco de dados. Assim, é interessante verificar se o sistema que escolhemos possui tal opção;
- **Controle de acesso** – permite definir o que cada usuário pode ver e/ou fazer/modificar. Dito de outra forma, podemos, com este controle, definir, por meio da necessidade de cada usuário, as suas permissões;
- **Estabilidade** – é necessário considerar o tamanho do banco de dados e do número de linhas nas tabelas. Assim, torna-se interessante conhecer alguns casos que se sucederam bem, a fim de podermos determinar se o *software* se enquadra às nossas necessidades;
- **Controle da redundância** – é essencial para um banco de dados garantir que seus dados não sejam repetidos. Com isso, não seria possível incluir dois registros com a mesma chave primária; e também não seria possível excluir um registro que tem relacionamento com outros;
- **Compartilhamento de dados** – é necessário que as informações estejam disponíveis para vários usuários de uma maneira rápida e segura, sendo este um requisito fundamental para um sistema.

Caso tenhamos aplicações que lidam com uma grande quantidade de consultas complexas, transações de banco de dados e análise de dados rotineiros, provavelmente será preferível utilizar um banco de dados relacional. E caso a sua aplicação tenha como

foco a execução de muitas transações, é importante que essas transações sejam processadas de forma confiável. Este cenário é ideal para o Acid – *Atomicity, Consistency, Isolation, Durability* –, conjunto de propriedades que garante que as transações de banco de dados sejam processadas de forma confiável, realmente importando onde a integridade referencial entra em jogo.

A integridade referencial é o conceito em que várias tabelas de banco de dados compartilham uma relação com base nos dados armazenados nas tabelas, de modo que tal relação deve ser coerente – isso geralmente é imposto com ações de adição, exclusão e atualização em cascata.



Figura 2 – Principais bancos de dados relacionais *open source*

Bancos de dados relacionais – *Relational Database Management Systems* (RDBMS) – tem sido o principal modelo para a gestão de banco de dados durante as últimas décadas. Contudo, nos últimos anos, bancos de dados NoSQL vem ganhando proeminência como um modelo alternativo de gestão de banco de dados.

O que Motivou o Surgimento do NoSQL?

Bancos de dados NoSQL abrangem uma grande variedade de diferentes tecnologias que foram criados em resposta às crescentes demandas apresentadas na construção de aplicações modernas.

Cada vez mais, são armazenados e gerados grandes volumes de tipos de dados e que mudam rapidamente, sejam dados estruturados, semiestruturados ou não estruturados. Em um passado não muito distante, o ciclo de desenvolvimento de aplicações era de doze a dezoito meses. Atualmente, pequenas equipes trabalham com metodologias ágeis de desenvolvimento, interagindo de forma rápida e liberando código a cada semana ou duas, algumas até mesmo várias vezes por dia.

Aplicações que antes serviam a determinado público finito, agora são entregues como serviços que devem estar sempre on-line, acessíveis a partir de vários dispositivos diferentes e disponíveis em todo o mundo para milhões de usuários.

As organizações têm reestruturado as suas arquiteturas e cada vez mais utilizam *software* de código aberto como, por exemplo, o Hadoop e a computação em nuvem, em vez de grandes servidores monolíticos e infraestruturas de armazenamento. Ou seja, vivenciamos uma transformação na forma como se coleta, armazena e analisa os dados e tal transformação requer inéditas tecnologias – exatamente daí surgiram as necessidades que têm levado ao crescimento de bancos de dados NoSQL.

O que são Bancos de Dados NoSQL?

NoSQL é uma tecnologia de banco de dados projetada para suportar os requisitos de aplicações em nuvem e arquitetado para superar o desempenho, escala, modelo de dados e as limitações de bancos de dados relacionais.

Os bancos de dados NoSQL – *Not Only SQL* – foram concebidos para armazenar, distribuir e acessar dados usando outras técnicas que diferem dos bancos de dados relacionais. A tecnologia NoSQL foi originalmente criada e usada por líderes da internet, tais como Facebook, Amazon e Google, além de outros que necessitavam de sistemas de gerenciamento de bancos de dados que pudessem escrever e ler dados em qualquer lugar no mundo e que garantissem desempenho a grandes conjuntos de dados e milhões de usuários.

Atualmente, quase todas as empresas e organizações possuem aplicativos em nuvem que personalizam a experiência do cliente com o seu negócio e soluções do tipo NoSQL, podendo ser a tecnologia de banco de dados escolhida para alimentar esses sistemas.

Você poderá encontrar uma lista completa de todos os bancos de dados NoSQL atualmente disponíveis em: https://goo.gl/mgPUru

A própria Oracle, líder mundial em bancos de dados relacionais, de olho nesse mercado, recentemente lançou a sua versão de banco de dados NoSQL.

Conheça-a em: https://goo.gl/dDJBP1



Figura 3 – Principais bancos de dados NoSQL *open source*

Qual Solução de Bancos de Dados Devemos Utilizar?

Primeiramente, precisamos considerar quais são as características dos dados que serão armazenados no banco de dados: se os dados tiverem estrutura tabular simples como, por exemplo, um cadastro de clientes, dados contábeis, folha de pagamento etc., o modelo relacional será o mais adequado.

Contudo, quando tratamos de dados geoespaciais, captação de dados através de sensores, peças de engenharia, modelagem molecular ou big data, referimo-nos a dados com estrutura mais complexa, ou seja, que podem apresentar vários níveis de aninhamento, de modo que o modelo completo de dados pode ser complicado. No passado, esses dados foram modelados em tabelas relacionais, mas não se encaixam perfeitamente em uma estrutura de linha-coluna bidimensional. Portanto, devemos considerar bancos de dados NoSQL como uma opção, pois possuem aninhamento e hierarquias multinível que são facilmente representadas no formato JSON – *JavaScript Object Notation* –, usado por alguns bancos de dados NoSQL, por exemplo.

Ademais, algumas perguntas precisam ser respondidas antes de decidir a melhor solução aos dados.

1. *Data ou big data?* Enquanto bancos de dados relacionais podem armazenar e manipular dados de forma confiável, a maior proposta para NoSQL vem na forma do que chamamos de big data, que são enormes quantidades de dados semiestruturados ou não estruturados gerados por máquinas ou pela sociedade todos os dias. A abordagem NoSQL, sem esquema de gerenciamento de dados, pode ser potencialmente a única maneira de entender como capturar, pesquisar, analisar e visualizar big data;
2. *Tabelas ou coleções?* Uma das principais diferenças entre bancos de dados relacionais e não relacionais é a forma como os dados são armazenados. Dados relacionais são tabulares por natureza, sendo armazenados em tabelas com linhas e colunas. As tabelas podem ser relacionadas entre si e cooperam no armazenamento de dados, bem como na fácil recuperação dos dados. Os dados não relacionais, por outro lado, não são destinados apenas à coleta em tabelas de linhas e colunas, mas sim agrupados em blocos. Os dados não relacionais são muitas vezes armazenados como coleções, tais como em documentos, pares chave-valor ou gráficos. Portanto, a natureza dos dados é um dos principais indicadores sobre a abordagem que melhor funciona ao armazenamento e à recuperação de dados;
3. *Schemas pré-definidos ou dinâmicos?* Os dados relacionais possuem dados estruturados, uma vez que as tabelas têm um esquema pré-definido para descrever os dados. Isso dá à modelagem de dados enorme importância e exige atitudes do tipo "faça certo da primeira vez". Enquanto um esquema pré-definido oferece confiabilidade e estabilidade, as mudanças a um esquema em uma tabela com dados pré-existentes é bem mais difícil. Os dados não relacionais são organizados em esquemas dinâmicos e, muitas vezes, são referidos como dados não estruturados. Os dados não relacionais podem facilmente acomodar mudanças no tipo e na estrutura de dados devido ao seu formato com esquemas dinâmicos;
4. *Acid ou CAP?* Bancos de dados relacionais são muito famosos por manterem a integridade por meio do que chamamos de Acid – *Atomic, Consistence, Isolated, Durable* –, propriedades encontradas em todos os bancos de dados relacionais, objetivando apoiar as operações indivisíveis isoladas cujas alterações são persistentes, deixando os dados em um estado consistente. Bancos de dados NoSQL, por outro lado, fazem com que você tenha que escolher entre quaisquer

duas prioridades do CAP – *Consistency, Availability, Partition tolerance* –, uma vez que todos os três são difíceis de se conseguir em um sistema baseado em nó distribuído;

5. *Consultas estruturadas ou não estruturadas?* Bancos de dados relacionais manipulam dados através de linguagem SQL, sendo incrivelmente poderosa para apoiar operações de *Crud – Create, Read, Update, Delete* –, um padrão no mercado de bancos de dados relacionais. Bancos de dados não relacionais manipulam dados em blocos – como documentos – e usam algo chamado *Unstructured Query Language* (UnQL), que não é padrão e pode variar entre os provedores de bancos de dados NoSQL. A chave primária em uma tabela relacional é equivalente ao conceito de ID de documento em um armazenamento não relacional. Os bancos de dados relacionais empregam otimizações pré-definidas, tais como definições de índice de coluna para ajudar a acelerar as operações de consulta, enquanto bancos de dados NoSQL desfrutam de padrões mais simples e restritos de acesso aos dados;

6. *Normalização ou custo de armazenamento?* O armazenamento de dados em bancos de dados relacionais aponta para maior normalização, tendo a necessidade de quebrar os dados em tabelas lógicas menores – relacionadas – para evitar a duplicação e obter melhor utilização do espaço. Embora a normalização dos dados seja fundamental, comumente adiciona um pouco de complexidade, especialmente em matéria de gestão de dados onde uma única operação pode abranger inúmeras tabelas relacionadas. Além disso, a normalização visava economizar o armazenamento de banco de dados, o que, em grande parte, é irrelevante no mundo atual, onde o custo de armazenagem – espaço em disco – é trivial. Por outro lado, os dados não relacionais possuem armazenamento em coleções planas, onde os dados podem muitas vezes ser duplicados. Um único bloco de dados raramente é dissociado, mas sim armazenado como uma entidade, permitindo, assim, acesso mais fácil de leitura e escrita a uma entidade única.

O que Pode Motivar a Utilização de Banco de Dados *Open Source*?

Em empresas que procuram reduzir o TCO – *Total Cost of Ownership* –, o mundo *open source* pode ajudar, mas alguns alertas devem ser levados em conta.

A escolha de um banco de dados *open source* se encaixa perfeitamente nesse cenário e em um momento no qual cortar custos é a palavra de ordem, de modo que o CIO – *Chief Information Officer* – pode recorrer à adoção desse modelo. Sem taxa de licenciamento, torna-se atrativo para as empresas, mas optar por esse modelo requer alguns cuidados que podem indicar não se tratar de boa decisão.

Ao adotar um banco de dados dessa modalidade, a empresa pode ser surpreendida pela necessidade de contratação de uma equipe de suporte técnico especializada e dedicada à solução escolhida. Isso requer um gasto considerável em contratações, treinamentos e na manutenção do produto, o que inclui a responsabilidade pelas instalações e atualizações necessárias para manter o produto "rodando". Além disso, a resolução rápida de problemas, como a indisponibilidade, pode ficar comprometida, já que as soluções precisam ser procuradas em fóruns de programação na internet, uma vez que os bancos de dados *open source* sofrem constantes alterações.

Existem também outros fatores que devem ser levados em consideração, tais como o código-fonte aberto, que diminui a segurança dos dados, ou as regras de licenciamento, que podem engessar o atendimento da demanda.

Assim, sempre é bom prestar atenção para duas perguntas sobre a relação custo *versus* benefício de um banco de dados *open source*:

Qual é a importância do seu negócio?

Quanto tempo você pode ficar com o seu negócio parado?

Não é difícil prever que a resposta dada por qualquer executivo é que o negócio é de extrema importância e que a solução deve auxiliar em seu crescimento – e não engessar a operação. Assim, não temos espaço para assumir riscos, de modo que a melhor opção é adotar um banco de dados fornecido por uma empresa especializada.

Ao adotar esse modelo, o custo-benefício é elevado, já que a taxa de licenciamento é compensada pela tranquilidade de ter atualizações e instalações realizadas pelo próprio fornecedor. Isso garante que o produto seja certificado e avaliado sob as melhores práticas do mercado, além de ser flexível ao melhor ajuste à necessidade do negócio. Além disso, despesas com suporte técnico já estão inclusas nos custos de manutenção, o que permite aos CIO saberem exatamente o quanto é investido e projetar quanto mais será necessário para manter o produto funcionando.

Outro ponto importante e que deve ser considerado ao adotar um banco de dados é a *performance*. Em *softwares* corporativos, priorize aqueles que contam com alta disponibilidade em ambientes *clusterizados* usando discos compartilhados. Dessa maneira, se um servidor cai, a solução se encarrega de utilizar outro, evitando que as tarefas sejam interrompidas.

Em suma, reduzir o TCO requer mais do que adotar aplicações que ofereçam uma mudança a curto prazo. É preciso cautela e controle de riscos. Por isso, ao considerar ajustes em suas ferramentas de Tecnologia da Informação (TI), principalmente a de banco de dados, opte por uma que garanta segurança, rapidez na solução de problemas e que ofereça uma infraestrutura completa e de acordo com a necessidade do negócio.

Bancos de Dados Relacionais *Versus* NoSQL: Qual Escolher?



Figura 4
Fonte: Adaptado de iStock/Getty Images

A melhor maneira para definir qual tecnologia de armazenamento devemos utilizar é observar os dados que serão armazenados:

Se os dados são tabulares, advindos de uma planilha, por exemplo, e atualizados frequentemente, ou ainda se o esquema dos dados é rígido, a escolha mais óbvia é utilizar um armazenamento relacional.

Caso exista a necessidade de transações Acid – Atomicidade, Consistência, Isolamento e Durabilidade –, tal característica existe em um SGBDR; se existe a necessidade de normalização ao máximo dos dados, esta normalização será mais performática em um SGBDR.

Se existe uma modelagem em grafos aos dados; se são hierárquicos; se o esquema de dados não está definido; se os tipos de dados são variáveis ou se a sua estrutura muda com frequência, o banco NoSQL seria a melhor escolha.

NoSQL é Mais Propício para Big Data

Analisando a escalabilidade do banco de dados, os SGBDR exigem um custo maior para a escalabilidade que comumente é vertical. A máquina servidor que geralmente é composta por um hardware mais robusto e, consequentemente, mais caro, exigirá *upgrade* a um custo bem alto.

Os bancos NoSQL que surgiram de empresas de internet altamente escaláveis, tais como *Amazon*, *Google* e *Facebook*, foram criados para rodar em máquinas com hardware comum e para serem linearmente escaláveis. A escalabilidade linear, onde são adicionadas novas máquinas em paralelo às já existentes, possui um custo igualmente linear, ou seja, muito menor em comparação ao da escalabilidade vertical.

Examinando a quantidade de dados armazenados, os SGBDR são bons para uma quantidade que consiga ser armazenada em uma ou poucas máquinas. Para big data, onde a quantidade de servidores pode ser de dezenas ou até centenas, bancos NoSQL se saem melhor. Os novos bancos NoSQL surgiram justamente com tal tarefa, armazenar uma quantidade imensa de dados utilizando dezenas ou centenas de servidores que empregam um hardware barato.

Os bancos NoSQL possuem capacidade de análise utilizando funções matemáticas e estatísticas. Porém, existe uma quantidade muito maior de ferramentas de relatórios e análise de dados para os SGBDR – e isto deve ser levado em consideração no projeto de banco de dados.

Finalmente, uma última análise é que aplicações centralizadas, como um Enterprise Resource Planning (ERP), geralmente são implementadas com mais facilidade em um SGBDR. Já aplicações descentralizadas, como web, móveis e de IoT – *Internet of Things* –, podem utilizar bancos NoSQL sem problema.

Eis algumas vantagens dos bancos NoSQL em relação aos SGBDR:

- Muito mais fácil e barato de administrar;
- A grande maioria é *software* livre, com custo de implantação bem menor;
- Escalabilidade elástica, fácil e mais barata;
- Armazena grandes volumes de dados;
- Desenhado para rodar em hardware de baixo custo;
- Reduz o tempo de desenvolvimento: não possui a necessidade de um modelo de dados ou normalização e o mapeamento entre objetos e documentos – para bancos NoSQL baseados em documentos – é mais direto que o mapeamento entre objetos e tabelas.

Já as desvantagens dos bancos NoSQL em relação aos SGBDR são as seguintes:

- Tecnologia nova, atualização rápida da tecnologia, comunidade nova, suporte falho, falta de documentação;
- Não possui a quantidade de ferramentas de relatórios, análises de desempenho, testes que os bancos de dados relacionais possuem;
- Não possui padrões de linguagem de definição e manipulação – ao contrário dos bancos de dados relacionais com o SQL, que é padrão;
- Falta de uma transação mais geral como nos SGBDR, as transações em bancos NoSQL são, geralmente, restritas a alterações em um único nó ou documento.

Conclusão

SGBDR e NoSQL são excelentes soluções em gerenciamento de dados e ambos são usados para manter o armazenamento e a recuperação de dados de forma otimizada. É difícil dizer qual tecnologia é melhor e, como sempre, o que determinará a escolha são os tipos de dados que você tem em mãos e a natureza da sua aplicação. Ademais, no seguinte Quadro você encontra algumas dicas gerais, a fim de lhe ajudar nesta escolha:

Quadro 1

Use RDBMS quando...	Use NoSQL quando...
Suas aplicações forem contralizadas (ERP, CRM)	Suas aplicações forem descentralizadas (Web, Mobile, Big Data, IoT)
Alta disponibilidade moderada for necessária	A disponibilidade tiver que ser contínua, sem interrupção
Dados gerados em velicidade moderada	Dados gerados em alta velocidade (sensores)
Dados forem gerados a partir de poucas fontes	Dados forem gerados a partir de múltiplas fontes
Dados dorem estruturados	Dados forem semi ou não estruturados
Transações complexas	Transações simples
For necessário manter moderado volume de dados	For necessário manter alto volume de dados

Os bancos NoSQL são novidades na tecnologia de armazenamento, que durante décadas girou apenas em torno dos bancos de dados relacionais. Festejado no início como um possível sucessor e substituto dos tradicionais bancos relacionais, atualmente podemos vislumbrá-lo mais como uma complementaridade.

Os sistemas atuais são extremamente complexos, com várias interfaces, acesso a bases de dados heterogêneas, com processamentos em batch e tempo real. As duas tecnologias de armazenamento de dados possuem papéis importantes em tal cenário, cada qual em um nicho específico e de acordo com a natureza dos dados e das respectivas características. Este parece ser o cenário ideal, com uma tecnologia complementando e auxiliando a outra, ambas resultando em um sistema final mais confiável e fácil de desenvolver.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Sites

MongoDB

https://goo.gl/N1k5Zq

NoSQL Data Base

https://goo.gl/7k4aUu

Vídeos

Introdução ao MongoDB

https://youtu.be/cmkretr_gOU

Introdução ao NoSQL - Curso de NoSQL | Devmedia

https://youtu.be/890222oPMmM

Leitura

A maior migração da história do Facebook: 30 petabytes e grandes desafios

https://goo.gl/dKE8WU

Bancos de dados não relacionais e o movimento NoSQL

https://goo.gl/SgGxfr

Referências

AMAZON. **DynamoDB**. [20--]. Disponível em: <https://aws.amazon.com/pt/dynamodb/m/pt/dynamodb>. Acesso em: 23 jan. 2018.

APACHE. **Cassandra**. [20--a]. Disponível em: <http://cassandra.apache.org>. Acesso em: 23 jan. 2018.

APACHE. **CouchDB**. [20--b]. Disponível em: <http://couchdb.apache.org>. Acesso em: 23 jan. 2018.

CLOUD. ***Bigtable***. [20--]. Disponível em: <https://cloud.google.com/bigtable/?hl=pt-br>. Acesso em: 23 jan. 2018.

DECANDIA, G. *et al. Dynamo: Amazon's highly available key-value store*. ***ACM Sigops Operating Systems Review***, v. 41, n. 6, p. 205-220, 2007.

ELMASRI, R.; NAVATHE, S. B.; DE OLIVEIRA MORAIS, R. **Sistemas de banco de dados**. [S.l.: s.n.], 2010.

HAN, J. *et al.* Survey on NoSQL database. *In:* PERVASIVE COMPUTING AND APPLICATIONS (ICPCA); INTERNATIONAL CONFERENCE ON (IEEE), 6., 2011. **Annales...** 2011. p. 363-366.

MARQUESONE. ***Big Data:*** técnicas e tecnologias para extração de valor de dados. São Paulo: Casa do Código, 2017.

MONGODB. [20--]. Disponível em: <https://www.mongodb.com>. Acesso em: 23 jan. 2018.

NEO4J. [20--]. Disponível em: <https://neo4j.com>. Acesso em: 23 jan. 2018.

PINTO, A. P. *et al.* Testes de performance utilizando o DB4O e MongoDB. **e-RAC**, v. 3, n. 1, 2013.

ROBINSON, I.; WEBBER, J.; EIFREM, E. ***Graph databases:*** *new opportunities for connected data*. [S.l.]: O'Reilly Media, 2015.

SADALAGE, P. J.; FOWLER, M. **NoSQL essencial:** um guia conciso para o mundo emergente da persistência poliglota. [S.l.]: Novatec, 2013.

VIEIRA, M. R. *et al.* Bancos de dados NoSQL: conceitos, ferramentas, linguagens e estudos de casos no contexto de Big Data. *In:* SIMPÓSIO BRASILEIRO DE BANCOS DE DADOS, 2012.

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000



Cruzeiro do Sul
Educacional